

## Задание 3. Данные, масштабирование и отказоустойчивость

### 1. Database per Service

| # | Сервис                                   | Тип базы данных                                                                                     | Обоснование                                                                                                                                                                                                                                                                                                                             |
|---|------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Сервис ставок<br>(Bidding Service)       | Redis (для горячих данных) + Собственная БД не используется, используется Read Replica для БД в п 2 | Redis нужен для сверхнизкой задержки (подгрузка правил аукциона, лимитов и пр) — критично для RTB<br><br>Используем Read Replica PostgreSQL сервиса рекламных кампаний используется для хранения неизменяемых правил кампаний, тарифов, историю ставок и служит источником истины. Такой подход даёт баланс скорости и консистентности. |
| 2 | Сервис кампаний<br>(Campaign Service)    | PostgreSQL                                                                                          | Данные о кампаниях — это транзакционные данные с жёсткими требованиями к целостности: статусы, бюджеты, расписания, таргетинги. PostgreSQL поддерживает ACID, сложные запросы по фильтрам и версиюемость изменений.                                                                                                                     |
| 3 | Сервис статистики<br><br>(Stats Service) | ClickHouse                                                                                          | Статистика — это массивные потоки событий (показы, клики, конверсии) с агрегациями по времени, гео локацией и т. п. ClickHouse оптимизирован под быстрые аналитические запросы на больших объёмах и высокую скорость записи. PostgreSQL для данной задачи, здесь будет слишком медленным при росте нагрузки.                            |
| 4 | Сервис аналитики<br>(Analytics Service)  | ClickHouse                                                                                          | Аналитика строится на исторических данных и сложных агрегациях. ClickHouse обеспечивает высокую производительность при работе с временными рядами и большими датасетами.                                                                                                                                                                |

|   |                                           |                                                |                                                                                                                                                                                                 |
|---|-------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | Сервис финансов (Billing/Finance Service) | PostgreSQL (строгая изоляция + WAL/репликация) | Финансы требуют максимальной надёжности, строгой консистентности и возможности аудита всех изменений. PostgreSQL гарантирует ACID транзакции, и возможность восстановления до точки во времени. |
|---|-------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 2. Масштабирование баз данных

### 2.1 Сервис ставок

Цель: держать P95 ≤ 80–100 мс при высоких RPS.

- Сервис не имеет собственной БД, используется Read replica PostgreSQL сервиса рекламных кампаний.
- Redis: репликация по умолчанию (MasterReplica) для отказоустойчивости. Для сверхвысокой доступности можно использовать Redis Cluster.

### CQRS

Частичный CQRS: поток на запись (идёт через Kafka. Чтение идёт напрямую из Redis. Это снимает нагрузку с мастер и позволяет обновлять лимиты без блокировок.

### 2.2 Сервис рекламных кампаний

Цель: целостность данных, поддержка сложных фильтров, умеренная нагрузка на запись.

#### Репликация и read replicas

- PostgreSQL MasterSlave: 1 мастер + read replica.
- Writes идут только на мастер; reads (списки кампаний, фильтры по гео/категориям) — на реплики.

### Шардирование

На начальном этапе не требуется.

При росте до значительного числа кампаний (например, более 1 м, в зависимости от метрик производительности): шардировать (например) таблицу рекламных кампаний по account\_id (или dsp\_id), чтобы кампании одного рекламодателя лежали на одном шарде. Это упрощает права доступа и отчёты по аккаунту.

## CQRS

Не обязателен для базовой CRUD логики.

### 2.3 Сервис статистики (Stats Service)

Цель: высокая пропускная способность на запись, быстрые агрегации по времени и атрибутам.

#### Репликация

ClickHouse: нативная репликация между нодами. Данные автоматически реплицируются по ZooKeeper/ClickHouse Keeper.

#### Шардирование

- Предлагается шардировать данные (таблицу) события показов/кликов по ключу времени `event_time` (партиционирование по дням) а также `ad_unit_id/campaign_id`.
- Типовой паттерн: партиции по дням, шарды по диапазонам `ad_unit_id`. Это ускоряет вставку и позволяет делать быстрые выборки по временным окнам.

## CQRS

Встроенный CQRS в ClickHouse: поток событий (raw) → агрегации (AggregatedMergeTree, SummingMergeTree) → витрины.

Raw - слой хранит все события; агрегированные таблицы — готовые срезы для API и дашбордов.

### 2.4 Сервис аналитики (Analytics Service)

Цель: сложные аналитические запросы, исторические данные, поддержка BI инструментов.

#### Репликация

ClickHouse: нативная репликация между нодами. Данные автоматически реплицируются по ZooKeeper/ClickHouse Keeper.

#### Шардирование

Аналогично сервису статистики: шардировать данные (таблицу) по `event_date` и шардирование по `campaign_id` либо по диапазонам ID.

Витрины (агрегации) могут иметь отдельную схему шардирования под частые запросы отчетов.

## Read replicas

Read replicas использовать для внешних (если будут) аналитических сервисов (Superset, PowerBI) делают много параллельных запросов. Реплики позволяют распределить эту нагрузку и не мешать потоковой записи.

- Можно выделить отдельный кластер или пул реплик только под BI-запросы.

## CQRS

Встроенный CQRS в ClickHouse: поток событий (raw) → агрегации (AggregatedMergeTree, SummingMergeTree) → витрины.

Raw - слой хранит все события; агрегированные таблицы — готовые срезы для API и дашбордов.

## 2.5 Сервис финансов (Billing/Finance Service)

Цель: максимальная надёжность, строгая консистентность, аудит, соответствие требованиям учёта.

### Репликация и read replicas

PostgreSQL MasterSlave (полусинхронная): 1 мастер, 2–3 реплики.

Все финансовые проводки (начисления, списания, корректировки) обращаются строго на мастер.

Read replicas используются для отчётов и сверки, чтобы не нагружать мастер.

### Шардирование

На начальном этапе не требуется.

Если объём транзакций станет очень большим: шардировать по `account_id + event_date`. Это сохраняет целостность операций внутри аккаунта и упрощает архивацию старых периодов.

## CQRS

Поток на запись напрямую PostgreSQL (в транзакции).

Поток чтения: отдельные таблицы/витрины для отчётов (сальдо, акты, сверки).

Такой подход позволяет отделить критичные проводки от тяжёлых отчётных запросов и обеспечить аудит всех изменений.

### 3. Кэширование

#### 3.1 Что кэшировать

| Тип данных            | Содержимое                                                    | Сервис                                   | Критичность                                                                 |
|-----------------------|---------------------------------------------------------------|------------------------------------------|-----------------------------------------------------------------------------|
| Правила кампаний      | Таргетинговые условия, ставки, приоритеты, креативные шаблоны | Сервис ставок, Сервис рекламных кампаний | <b>Высокая</b> – отсутствие приведёт к невозможности проведения аукциона    |
| Бюджеты кампаний      | Общий и суточный остаток бюджета                              | Сервис рекламных кампаний                | <b>Высокая</b> – неактуальные данные → overspend или недополучение дохода   |
| Профили пользователей | Сегменты аудитории, frequency capping, история показов        | Сервис ставок, Сервис рекламных кампаний | <b>Высокая</b> – влияет на таргетинг и частоту показов                      |
| Баннеры               | HTML/JS-код, изображения, ссылки (или ссылки на CDN)          | Сервис выдачи                            | <b>Средняя</b> – при отсутствии используется fallback или генерация на лету |

#### Размещение Redis

- Redis Cluster размещается в том же датацентре (или availability zone), что и основные сервисы RTB-пути (Bidding Ingress, Auction Engine), чтобы минимизировать сетевую задержку (< 1 мс).
- Используется Redis Cluster с несколькими мастер-нодами и репликами (минимум 3 мастера, каждый с репликой) для обеспечения высокой доступности и горизонтального масштабирования.

- Размер кэша начально — 64 ГБ памяти на каждый инстанс кластера, с возможностью расширения.

### 3.2 TTL и стратегия инвалидации

| Тип данных                        | Рекомендуемый TTL | Обоснование                                                              |
|-----------------------------------|-------------------|--------------------------------------------------------------------------|
| Счётчики частоты (hot counters)   | 5–15 минут        | Данные быстро устаревают; пересчёт лимитов делается по событиям из Kafka |
| Правила аукциона, лимиты кампаний | 1–5 минут         | Должны быть свежими; частые обновления через события                     |
| Карточки кампаний (общие данные)  | 10–30 минут       | Баланс между актуальностью и нагрузкой на БД                             |

### 3.3 Стратегии инвалидации

**По событию (Event-driven)** (рекомендуется для ставок и лимитов):

- При изменении лимита/статуса в PostgreSQL, публикуется соответствующее событие в Kafka.
- Consumer обновляет/удаляет соответствующие ключи в Redis. Это даёт согласованность без долгих TTL.

**По истечению времени жизни (TTL-based)**: для справочников и других данных, где допустима задержка.

**Явная инвалидация (Explicit invalidation)**: при ручных изменениях или действий администраторов (например, «остановить кампанию») сразу удаляем/обновляем ключи.

**Также возможна комбинация методов**: для критических данных (лимиты, счётчики) использовать события + короткий TTL как страховку. Для справочников — события или длинный TTL.

### 3.4 Прогрев кэша (cache warming)

Прогрев нужен, чтобы избежать «холодного старта» и всплесков нагрузки на БД при рестарте Redis или деплое.

**Подход 1**: фоновый воркер (background warmer)

- При старте Redis запускается воркер, который делает выборки «горячих» кампаний (по количеству показов, по бюджету, по активности).
- Данные загружаются в Redis с теми же ключами и TTL, что и в runtime.
- Пример: топ1000 активных кампаний + топ500 DSP.

**Подход 2:** прогрев по трафику (ondemand + prefetch)

- Bidding Service при старте делает лёгкие выборки последних активных кампаний и заранее кладёт их в кэш.
- Можно использовать скользящее окно «последние N часов» по событиям из Kafka.

## 4. Поточковая обработка данных

Поточковая обработка событий отделяет синхронный RTB-путь (аукцион) от асинхронных бизнес-процессов (финансы, статистика, обновление профилей, аналитика).

В качестве центральной шины используется Apache Kafka с Avro-сериализацией и Schema Registry.

### 4.1 Топики Kafka

| Топик               | Ключ           | Назначение                                                                                                                                                                                            |
|---------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rtb.bid-won         | auction_id     | Публикуется сервисом рекламных кампаний после каждого выигранного аукциона. Содержит campaign_id, user_id, win_price, timestamp, targeting_context. Используется для биллинга, статистики, инвентаря. |
| tracking.impression | impression_id  | Фиксирует показ рекламы. Содержит auction_id, creative_id, user_id, viewability_data. Генерируется сервисом ставок после ответа.                                                                      |
| tracking.click      | click_id       | Фиксирует клик по рекламе. Содержит impression_id, click_url, timestamp.                                                                                                                              |
| campaign.updated    | campaign_id    | Оповещает об изменении кампании (бюджет, таргетинг, ставки, статус). Используется для обновления кешей (Redis) и пересчёта инвентаря.                                                                 |
| finance.transaction | transaction_id | Финансовые операции (списания, пополнения). Содержит user_id, amount, type, status. Для аудита и синхронизации.                                                                                       |
| cdc.campaigns       | campaign_id    | изменения таблиц кампаний.                                                                                                                                                                            |
| cdc.finance         | transaction_id | Сырые изменения финансовых записей.                                                                                                                                                                   |



## 4.2. Схема событий: Avro

**Avro (рекомендуется как основной вариант):** сырые события (показы, клики, конверсии), ставки, лимиты — всё, где важна производительность, строгая схема и эволюция версий.

### Плюсы:

- Компактность (бинарный формат, меньше сетевой трафик и дискового места).
- Строгая схема + версионирование (Schema Registry) — защита от «битых» событий.
- Быстрая сериализация/десериализация.
- Централизованное управление схемами через Schema Registry, валидация совместимости.

### Минусы:

Сложнее отлаживать «в лоб» (нужен Schema Registry и инструменты).

### Базовые поля (в каждом событии):

avro

```
{
  "type": "record",
  "name": "BaseEvent",
  "fields": [
    {"name": "event_id", "type": "string"},
    {"name": "timestamp", "type": "long"},
    {"name": "source_service", "type": "string"},
    {"name": "trace_id", "type": "string"},
    {"name": "schema_version", "type": "int"}
  ]
}
```

Каждое событие расширяет эти поля специфическими атрибутами.

## 4.3. Группы потребителей

Каждый микросервис подписывается как отдельная группа, что позволяет параллельно читать одни и те же топики без влияния на смещения других групп.

| Группа потребителей | Топики   | Назначение                       |
|---------------------|----------|----------------------------------|
| stats-service-      | rtb.bid- | Агрегация и запись в ClickHouse. |

|                         |                                                                   |                                                                            |
|-------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------|
| group                   | won, tracking.impression, tracking.click                          |                                                                            |
| billing-service-group   | rtb.bid-won, finance.transaction                                  | Запуск саг по списанию средств.                                            |
| inventory-service-group | rtb.bid-won, campaign.updated                                     | Обновление материализованных представлений инвентаря.                      |
| profile-service-group   | tracking.impression, tracking.click, campaign.updated             | Обогащение профилей поведенческими данными.                                |
| analytics-service-group | tracking.impression, tracking.click                               | Построение дашбордов (опционально, если не читает из ClickHouse напрямую). |
| cache-updater-group     | cdc.campaigns, cdc.users, campaign.updated, user.profile.enriched | Обновление кеша Redis и локальных кешей.                                   |

#### 4.4. Политика хранения (Retention)

| Топик               | Retention (время) | Retention (объём) | Обоснование                                                                   |
|---------------------|-------------------|-------------------|-------------------------------------------------------------------------------|
| rtb.bid-won         | 7 дней            | 500 GB            | восстановление после сбоев.                                                   |
| tracking.impression | 7 дней            | 1 TB              | Аналогично.                                                                   |
| tracking.click      | 7 дней            | 500 GB            | Аналогично.                                                                   |
| campaign.updated    | 14 дней           | 100 GB            | Сохранение последнего состояния кампании для новых потребителей (compaction). |
| finance.transaction | 30 дней           | 200 GB            | Финансовый аудит.                                                             |

## 5. Отказоустойчивость данных

### 5.1 RPO и RTO по сервисам

| Сервис            | RPO                            | RTO        | Обоснование                                                                                                                                                                                                                                                                                                                  |
|-------------------|--------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Сервис ставок     | ≤ 5 минут (для логов аукциона) | ≤ 15 минут | Данные хранятся в Redis и локальном кеше, которые могут быть перестроены из PostgreSQL через CDC. Основная потеря – это логи аукциона, которые записываются асинхронно через Kafka в БД сервиса рекламных кампаний.                                                                                                          |
| Сервис кампаний   | ≤ 15 минут                     | ≤ 1 час    | Данные кампаний хранятся в PostgreSQL. Изменения не критичны с точностью до минут – допускается небольшая потеря при сбое, но важно восстановить последнюю согласованную версию. RTO учитывает время восстановления из резервной копии и применения WAL.<br><br>Лучшее решение – использование кластера Postgres SQL Patroni |
| Сервис статистики | ≤ 15 минут                     | ≤ 4 час    | Сырые события (клики/показы) буферизируются в Kafka с retention 7 дней, поэтому потеря данных может быть компенсирована репроцессингом. RPO определяется задержкой между записью в ClickHouse и последним снэпшотом. RTO – время восстановления ClickHouse из реплик/снэпшотов.                                              |
| Сервис аналитики  | ≤ 1 час                        | ≤ 4 часа   | Данные являются агрегированными и не критичны для операционной работы. Допустима потеря до часа, т.к. дашборды могут быть перестроены из сырых данных статистики. RTO больше из-за возможной сложности восстановления больших OLAP-таблиц.                                                                                   |
| Сервис финансов   | ≤ 5 минут                      | ≤ 30 минут | Финансовые транзакции критичны. Используется синхронная репликация и частые WAL-архивации. RPO минимизирован до 5 минут. RTO 30 минут для минимизации влияния на бизнес.<br><br>RTO учитывает время восстановления из резервной копии и применения WAL.<br><br>Лучшее решение – использование                                |

|  |  |  |                               |
|--|--|--|-------------------------------|
|  |  |  | кластера Postgres SQL Patroni |
|--|--|--|-------------------------------|

## 5.2. Стратегии резервного копирования

| Сервис                       | Частота полных бэкапов | Инкрементальный бэкап /логи транзакций | Хранение                        | Восстановление                                                               |
|------------------------------|------------------------|----------------------------------------|---------------------------------|------------------------------------------------------------------------------|
| Сервис кампаний (PostgreSQL) | Ежедневно (ночью)      | WAL каждые 5 минут                     | S3 (30 дней полные, 7 дней WAL) | Восстановление до последней точки во времени с применением WAL.              |
| Сервис финансов (PostgreSQL) | Ежедневно (ночью)      | WAL каждые 5 минут                     | S3 (до года)                    | Восстановление до последней точки во времени с применением WAL.              |
| ClickHouse (Статистика)      | Ежедневно (ночью)      | Репликация                             | S3                              | Восстановление из снапшотов или переключение на реплику.                     |
| ClickHouse (Аналитика)       | Раз в 2 дня            | Репликация                             | S3                              | Восстановление из снапшотов, при необходимости перестроение из сырых данных. |

### Дополнительно:

Ежемесячно проводится **тестовое восстановление** в изолированной среде для проверки целостности бэкапов и времени восстановления (RTO).

## 5. 3. Соответствие принципам 12-факторных приложений

Архитектура спроектирована с учётом 12-факторной методологии, что обеспечивает переносимость, масштабируемость и упрощает управление конфигурациями.

| Фактор       | Реализация в AdScale                                                                                                                 |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Кодовая база | Каждый микросервис имеет собственный репозиторий; один репозиторий – один сервис. Мониторинг и конфигурация – отдельные репозитории. |

|                                              |                                                                                                                                                                                                   |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Зависимости                                  | Все зависимости явно объявлены, контейнеризация (Docker) изолирует окружение.                                                                                                                     |
| Конфигурация                                 | Вся конфигурация (URL-адреса БД, Redis, Kafka, топики, параметры TTL) хранится в переменных окружения и/или в Consul/Vault.                                                                       |
| Сторонние службы                             | Все внешние сервисы (БД, очереди, кэши) подключаются через URL-адреса, задаваемые через переменные окружения. Это позволяет легко переключать окружения (dev/stage/prod).                         |
| Сборка, релиз, запуск                        | Используется CI/CD (GitLab CI) – сборка образа (build), затем деплой (release) в Kubernetes с использованием Helm-чартов. Строгое разделение этапов.                                              |
| Процессы                                     | Сервисы stateless – состояние вынесено во внешние хранилища (PostgreSQL, Redis). Это позволяет горизонтально масштабировать, убивать и перезапускать процессы без потери данных.                  |
| Порт привязки                                | Каждый сервис экспортирует свой порт (например, 8080 для REST, 9090 для gRPC). В Kubernetes это декларируется в Service/Ingress.                                                                  |
| Конкурентные процессы                        | Масштабирование через реплики в Kubernetes (Horizontal Pod Autoscaler). Каждый процесс обрабатывает запросы конкурентно; для асинхронной обработки используются пулы воркеров и Kafka-консьюмеры. |
| Утилизируемость                              | Быстрый запуск (< 5 секунд) и graceful shutdown (обработка SIGTERM с корректным завершением). Это важно для обновлений и автоматического восстановления.                                          |
| Журналы (логи)                               | Все логи выводятся в stdout/stderr и собираются в централизованное хранилище (ELK). Это позволяет анализировать логи без доступа к файловой системе контейнера.                                   |
| Административные процессы                    | Все админ задачи выполняются как разовые (отдельные Kubernetes Jobs) или через вызовы API, а не в основном процессе.                                                                              |
| Среды разработки тестирования и эксплуатации | Для каждой среды используется отдельная инфраструктура на общем техстеке а также отдельные инсталляции системы . Отсутствие общих компонентов для каждой среды, полное разделение                 |